Java Spring · Week 2

Logs, and who builds your objects

logback · the Spring context · beans

Eng. Mohamed Shehab

Faculty of Computers and Information, Menoufia University

Where we stopped last week

You pick the level. A file picks the threshold.

In your code	In logback.xml
`log.debug("...")`	`<root level="INFO">`
You choose a level for every single line, while you write it.	You choose one threshold for the whole application, at run time.
Changing it means editing code, building, and deploying.	Changing it means editing a file and restarting. No build.

Today we finish the job: how to write the line itself, and where to put it.

The one habit worth building today

Braces, not plus

Do this

```
log.debug("{} seats left for {}",
 available, event);
```

The string is built inside the library, and only if the level is on.

Not this

```
log.debug(available +
 " seats left for " + event);
```

Java builds the string first, then hands it to a method that may throw it away.

With DEBUG off and a million rows: zero strings built, or a million built and binned.

Two mistakes that cost you hours later

Logging a failure

```
log.error("could not send confirmation for {}", event, e);
```

The rule	Why
The exception goes last, with no brace of its own	SLF4J works that out by itself and prints the whole stack trace
Never `e.getMessage()` on its own	You learn that something failed, never which file and which line
A refused request is `warn`, not `error`	If everything is an error, a real error stops meaning anything

In a real project you never open DEBUG on everything

Turn it up for your code only

```
<logger name="org.example" level="DEBUG"/>
<root level="WARN"/>
```

Your classes talk. Hibernate, Tomcat and Spring stay quiet.

Level	Put one here
info	A booking went through. A job started. Something a human may ask about later.
warn	A retry, a cache miss, a fallback. It worked, but not the way you planned.
error	This request died. Always with the exception attached.

The line you just typed, and one word that writes it for you

An annotation is a note stuck on your code

You wrote this

```
private static final Logger log =
 LoggerFactory.getLogger(Main.class);
```

Delete it, write this instead

```
@Slf4j
public class Main { ... }
```

javap -p target/classes/org/example/Main.class

```
public class org.example.Main {
 private static final org.slf4j.Logger log; ← still there
}
```

Gone from your file, present in the compiled one. Something read the word and wrote the line.

`@Override`	`@Test`	`@Entity`	`@Component`
the compiler	JUnit	Hibernate	Spring, in a moment

One word wrote one line. Here is one that writes ten methods.

Same idea, bigger payoff

```
@Data // three fields, zero methods
public class Booking { ... }
```

`@Getter @Setter`

six methods appear: three getters, three setters

`@Data`

those six, plus toString, equals, hashCode and canEqual

```
// with @Getter @Setter
the booking: org.example.Booking@45e5696d

// with @Data
the booking: Booking(user=ahmed@example.com, eventTitle=Spring Boot Workshop, seats=5)
```

The part nobody sees is the bigger part

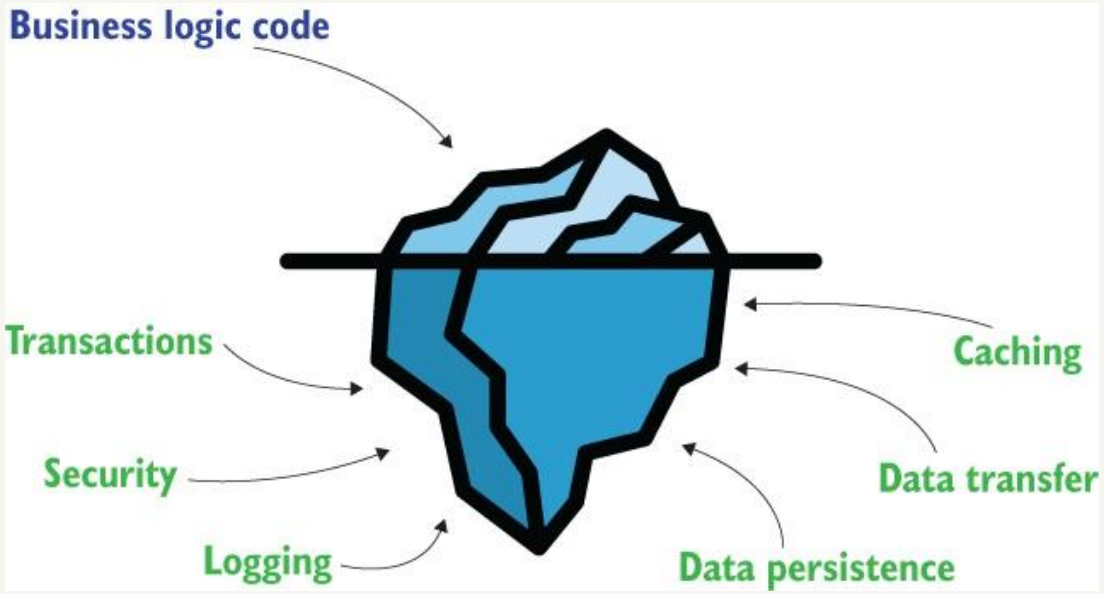


figure 1.2 · Spring Start Here, Laurentiu Spilca, Manning Publications

You spent the last hour on one item under the water. Every app needs all of them, and none of them is your product.

Spring is not a framework. It is an ecosystem of them.

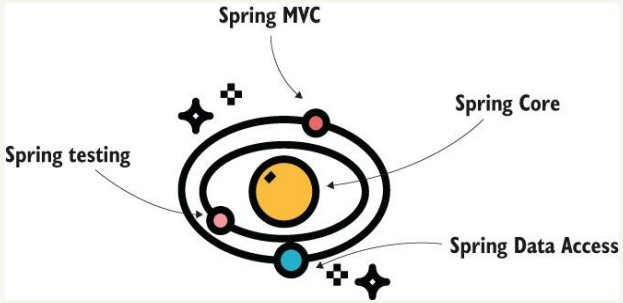


figure 1.3 · Spring Start Here, Laurentiu Spilca, Manning Publications

Inside the Spring framework	What it is for
`Spring Core`	the context, aspects, the foundations. today
`Spring MVC`	web apps that answer HTTP
`Spring Data Access`	JDBC, ORM, transactions
`Spring testing`	tools for writing tests
and whole projects around it	Spring Boot, Spring Security, Spring Cloud, Spring Batch, and more

Out of all of that, here is our term

Where this section sits

The piece	Where it shows up	Week
`Spring Core` · the context	beans, wiring, scopes, the container	2 and 3 · here
`Spring Boot`	convention over configuration	4
`Spring MVC`	REST controllers, validation, layers	5 and 6
`Spring Data`	JPA, relations, transactions, queries	7 to 9
the wider ecosystem	Redis, then RabbitMQ	10 and 11
`Spring testing`	tests, and Docker alongside them	12
`Spring Core` · aspects	chapter 6 of the book	not in this course

It is built on top of itself. If today is unclear, everything after it is unclear.

Chapter 1 · section 1.4, the part most courses skip

And when not to use one

- The app has to stay tiny, like a serverless function
- Security rules say custom code only
- You would customise it more than you would write without it
- The app already works and you gain nothing

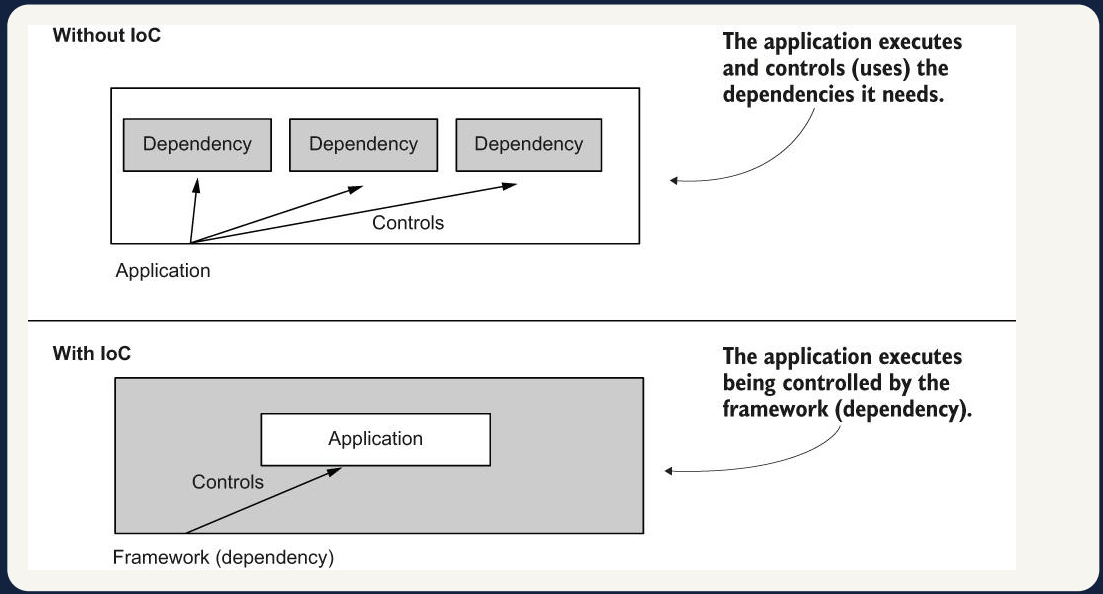


figure 1.4 · Spring Start Here, Laurentiu Spilca, Manning Publications

Nothing fits every case. Next: the one idea in that second picture.

Week 1 · you wrote this yourself

Who writes new for you?

```
ObjectMapper mapper = new ObjectMapper();
```

One object, one line. Fine. Now the real thing:

```
new BookingService(
 new SmtpEmailSender(
 new MailConfig(host, port, user)))
```

Thirty services, in the right order, built once and shared. Someone has to own that job.

Before anything goes in it, you need the box

The Spring context

```
var context = new AnnotationConfigApplicationContext(AppConfig.class);
```

`Annotation`
the configuration comes from notes on classes, not from an XML file

`Config`
and the class holding those notes is `AppConfig`

`ApplicationContext`
the box itself. One object that holds your objects

From week 4, Spring Boot writes this line for you. You will not see it again, so look at it now.

config/AppConfig.java · one note on it, and nothing inside

And it starts empty

```
@Configuration
public class AppConfig {
}
```

```
beans in the context: 5
getBean(Event.class) -> No qualifying bean of type
 'com.eventia.model.Event' available
new Event(1L, "my own event", 10);
beans in the context: 5 ← it did not move
```

Four of those five are Spring's own. The fifth is `AppConfig` itself: a configuration class is a bean too.

2.1 in the book · one method, one note on it

You put the first one in yourself

```
@Bean
public Event workshop() {
 return new Event(1L, "Spring Boot Workshop", 40);
}
```

```
beans in the context: 6 ← was 5
 appConfig
 workshop ← the method name
got it: Spring Boot Workshop with 40 seats
```

You wrote a method. You never call it.

What Spring did with it

1

It called your method

once, while the context was starting up

2

It kept what came back

that object now lives in the box

3

It named it after the method

`workshop`, not `Event`

```
@Data // Lombok, the one you just met
public class Event { ... } // and not one Spring annotation
```

`a service`

the booking logic

`a repository`

the thing that talks to the database

`an email sender`

one connection, shared

`a JSON mapper`

one shared, set up once

Ask them before you run it: which one comes back?

Two of them. Spring stops.

```
@Bean
public Event conference() {
 return new Event(2L, "Java Conference", 200);
}
```

```
beans in the context: 7
getBean(Event.class) ->
 No qualifying bean of type 'com.eventia.model.Event' available:
 expected single matching bean but found 2: workshop,conference
```

It will not guess. A silently wrong answer is worse than a stop.

The first way out: say which one

Ask for it by name

```
context.getBean("workshop", Event.class); // Spring Boot Workshop
context.getBean("conference", Event.class); // Java Conference
```

And the name is just the method name, so you can change it.

```
@Bean("mainEvent")
public Event workshop() { ... }
```

```
mainEvent ← the method name is gone
 conference
getBean("workshop") -> No bean named 'workshop' available
```

The second way out: do not say the name at all

Mark one as the default

```
@Bean
@Primary
public Event workshop() {
 return new Event(1L, "Spring Boot Workshop", 40);
}
```

```
getBean(Event.class) now works -> Spring Boot Workshop
```

by name

you decide every time you ask, in the asking code

@Primary

you decide once in the config, and callers need not know there was a choice

Clock belongs to the JDK. You cannot write a note on it.

The classes that are not yours

```
@Bean
public Clock clock() {
 return Clock.systemUTC();
}
```

```
beans in the context: 8 ← was 7
 clock ← the method name again
got a class I do not own: java.time.Clock$SystemClock
```

a DataSource
the database connection pool. Week 7

an HTTP client
for calling another service

ObjectMapper
the one you used in week 1

Week 1 you wrote `new ObjectMapper()` by hand. Same problem, same door.

Your own classes. Is there a shorter way?

Four methods became one word

what you had

```
@Bean
public Event workshop() {
 return new Event(...);
}

@Bean
public Event conference() {
 return new Event(...);
}
```

what you write instead

```
@Component
public class BookingService { ... }


@Configuration
@ComponentScan("com.eventia")
public class AppConfig {
}
```

Only for classes you own. `Clock` stays a `@Bean`, in the same config, next to this.

None of them runs. Each one is data Spring looks for.

The notes Spring reads

The note	What Spring does when it reads it
`@Component`	build one object of this class and keep it
`@ComponentScan`	walk this package and find the notes yourself
`@Service`	the same as @Component, with a label on it

```
// Service.class.getAnnotations(), measured
@Target({TYPE})
@Retention(RUNTIME)
@Documented()
@Component("") ← @Service IS a @Component
```

config/AppConfig.java · the @Bean methods are gone

You hand over no list

```
@Configuration
@ComponentScan(basePackages = "com.eventia")
public class AppConfig {
}
```

1
It opens the package
the one, and only the one, you named

2
It looks at every class
is there a note on it, or not

3
It builds the ones there are
and that is the whole trick

You said where to look. You never said what is in there. Add a class tomorrow and this file does not change.

Two ways to break it, and they look the same from outside

Same message, two different causes

1 · take the note away

```
public class BookingService {
 ...
}

// the @Component line is gone
```

2 · keep it, look elsewhere

```
@ComponentScan(basePackages =
 "com.other")

// the note is still on the class
```

```
No qualifying bean of type 'com.eventia.service.BookingService' available
```

One message. Twice. The note is not enough on its own: the class has to be where Spring is looking.

The question that asks itself

Same box, two doors

`@Bean`	**stereotype**	
`@Bean` on a method in the config	`@Component` on the class	where the note goes
a whole method per bean	one word	how much you write
yes	no	a class you do not own
yes	no	several of the same type
only when you must	everything else	what we use from here on

Run it. Eight became six, and the code got shorter.

One word did what four methods did

```
INFO c.e.s.BookingService - ahmed@example.com booked a seat
INFO com.eventia.App - beans in the context: 6
INFO com.eventia.App - bean name: bookingService ← from the class name
INFO com.eventia.App - same instance? true
```

6 beans

four are Spring's own, one is AppConfig, one is yours

bookingService

the class name, not a method name. That is the last practical difference between the two doors

same instance

asked twice, got the same object. A singleton

One log line inside the constructor, and three in App

Built before you asked

```
public BookingService() { log.info("constructor ran"); }
```

```
INFO com.eventia.App - building the context now
INFO c.e.s.BookingService - constructor ran
INFO com.eventia.App - context ready, I asked for nothing yet
INFO com.eventia.App - now I asked
INFO c.e.s.BookingService - ahmed@example.com booked a seat
```

line 2 beat line 3

Spring builds its singletons while the context starts, not when you ask

and never again

getBean built nothing. It looked one up.

One thing we did not do today

Our service needed nothing

```
@Service
public class BookingService {

 private final EmailSender emailSender;

 BookingService(EmailSender emailSender) { ... }
```

Next week it needs an `EmailSender`, and Spring hands one in. That is dependency injection.

Five things to take away

1. The level is chosen in the code. The threshold lives in a file outside it.
2. Braces, never plus. And the exception goes last, with no braces of its own.
3. error means something of mine is broken. A refused request is warn.
4. An annotation is data. Nothing happens until something reads it. Spring is that something.
5. You never wrote new. The container built it once and handed you the same instance.

Next week

Dependency injection: Spring passes in what your service needs

Eng. Mohamed Shehab

Faculty of Computers and Information, Menoufia University

mohamed.shehab.fci@gmail.com